

Software Engineering with Generative AI

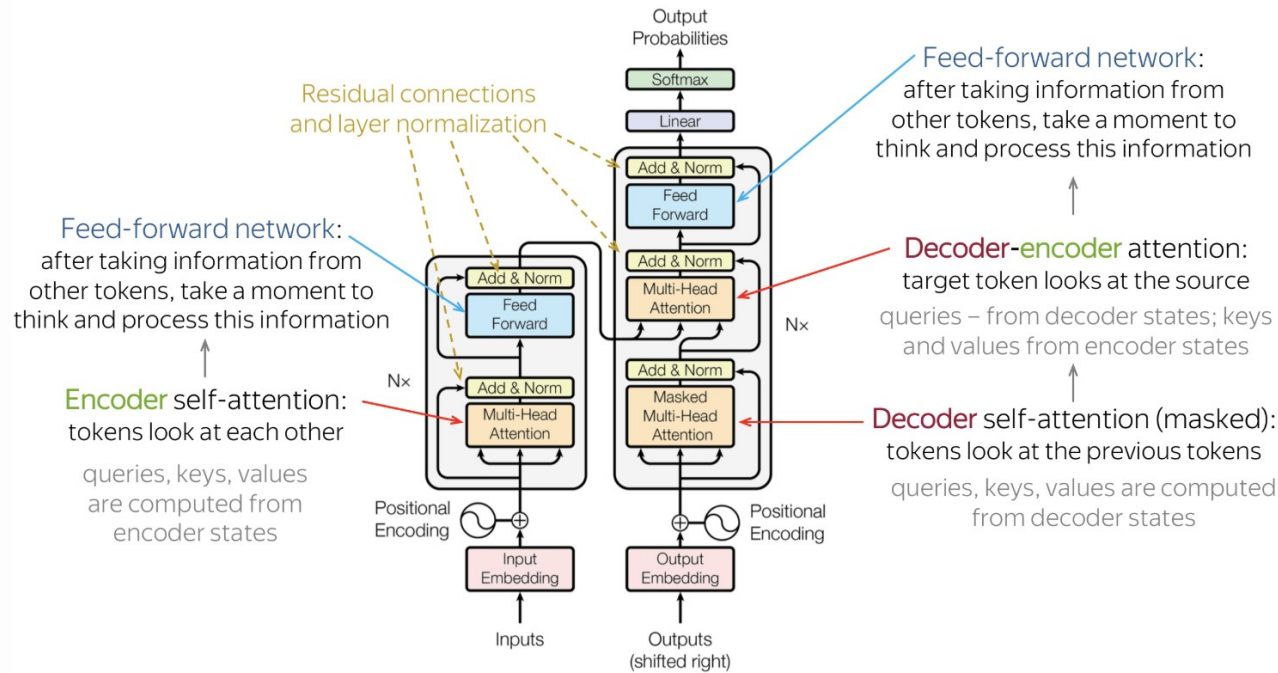
Week 3 · Transformer, Decoding, KV cache, prompt engineering
Instructor: Jie “JW” Wu
CS 5090 · Fall 2026
Michigan Technological University



Transformer Architecture

we have a few things left:

- masked/causal self-attention
- position encoding
- residual connection
- layer normalization
- FFN
- We'll cover them later...



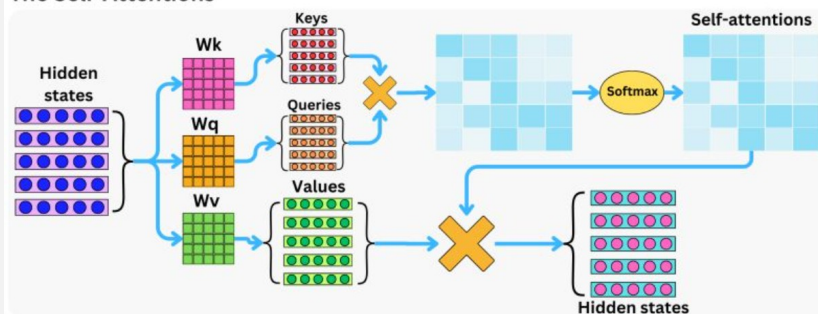
Source: https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html



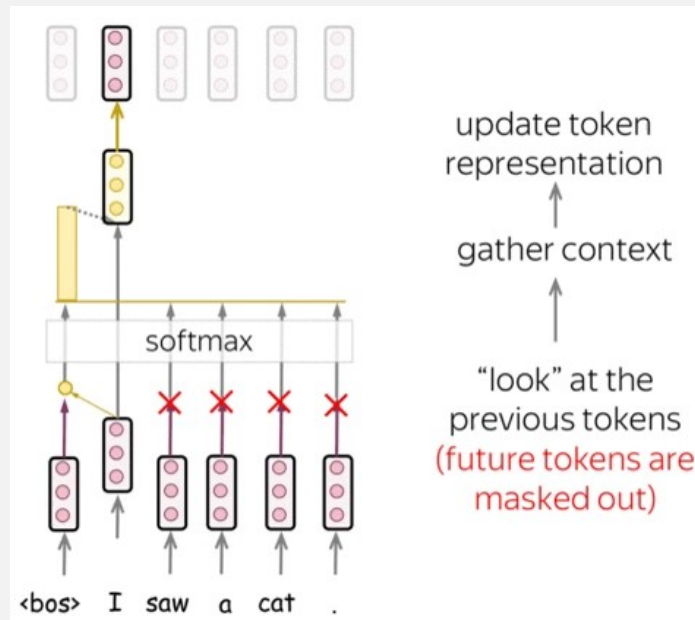
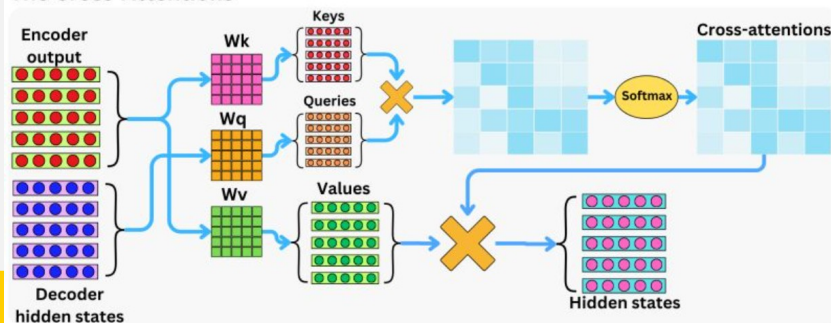
Self vs cross vs masked attention

Self-Attention VS Cross-Attention *TheAiEdge.io*

The Self-Attentions



The Cross-Attentions

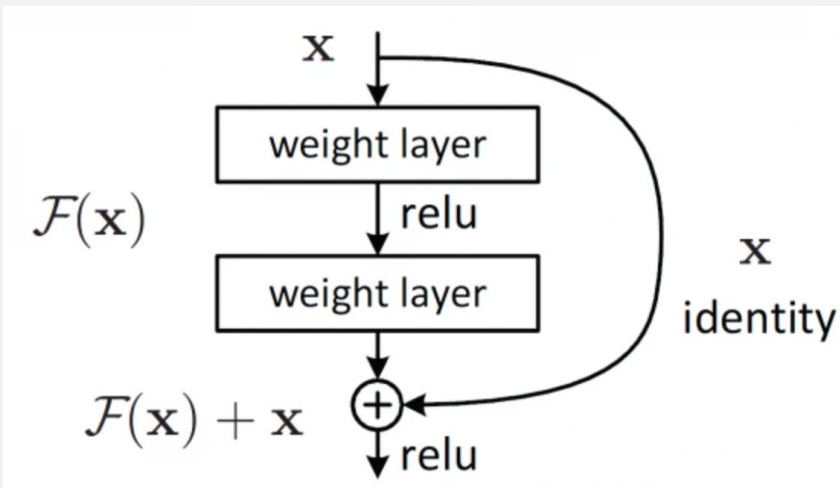


Source:

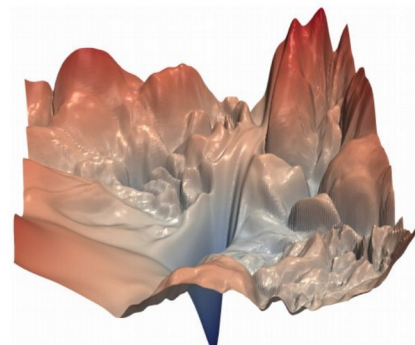
https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html



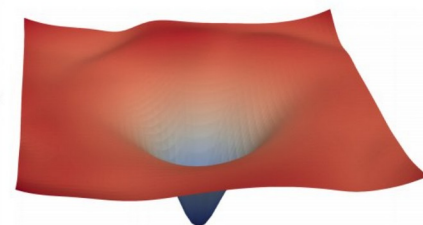
Residual Connection



<https://arxiv.org/abs/1512.03385>



(a) without skip connections



(b) with skip connections

Figure 1: The loss surfaces of ResNet-56 with/without skip connections. The vertical axis is logarithmic to show dynamic range. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures.

Source: N. Krishnamoorthy & Vignesh Raaja

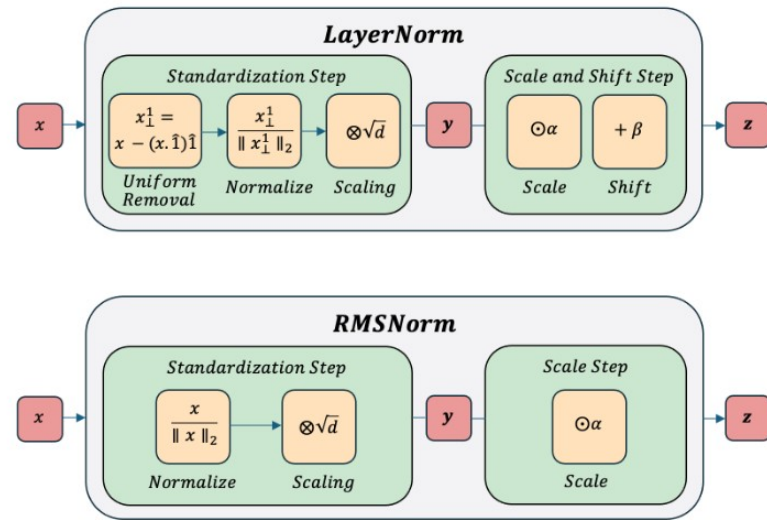
Layer Norm

Layer normalization helps models train faster.

LayerNorm steps:

1. Compute the mean across hidden dimensions.
2. Subtract the mean (centering).
3. Compute the variance.
4. Divide by the standard deviation (normalization).
5. Apply learnable scale and shift: γ and β .

$$y = \gamma \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$



(a) A diagrammatic explanation of LayerNorm and RMSNorm.

Layer norm vs batch normalization?

Source:

<https://arxiv.org/pdf/2409.12951v1>

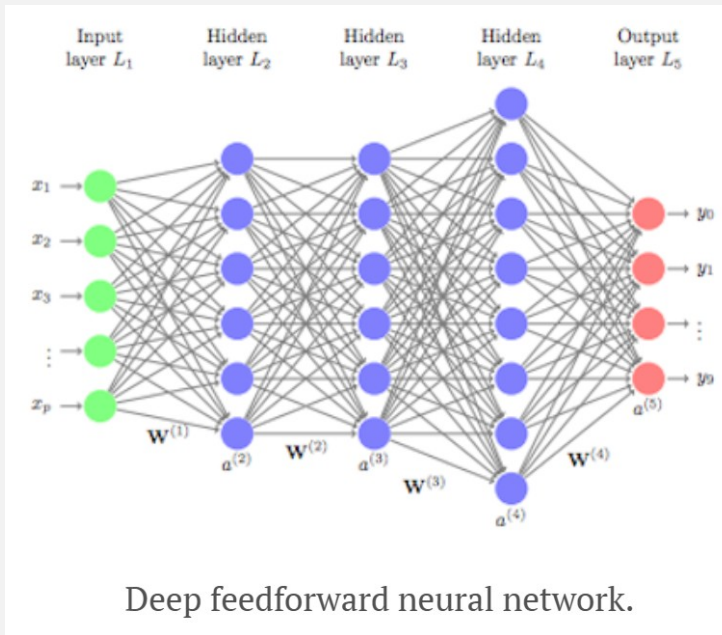


Feed Forward Network

why need that? hint: non-linearity, need processing

Attention: lets one token gather information from other tokens.

FFN: processes and transforms the information inside each token.



Source:

https://afit-r.github.io/feedforward_DNN

Position Encoding

Why not learn position embedding from training data?

- Dependent on training set

From transformer original paper:

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

// We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset k , PE_{pos+k} can be represented as a linear function of PE_{pos} .

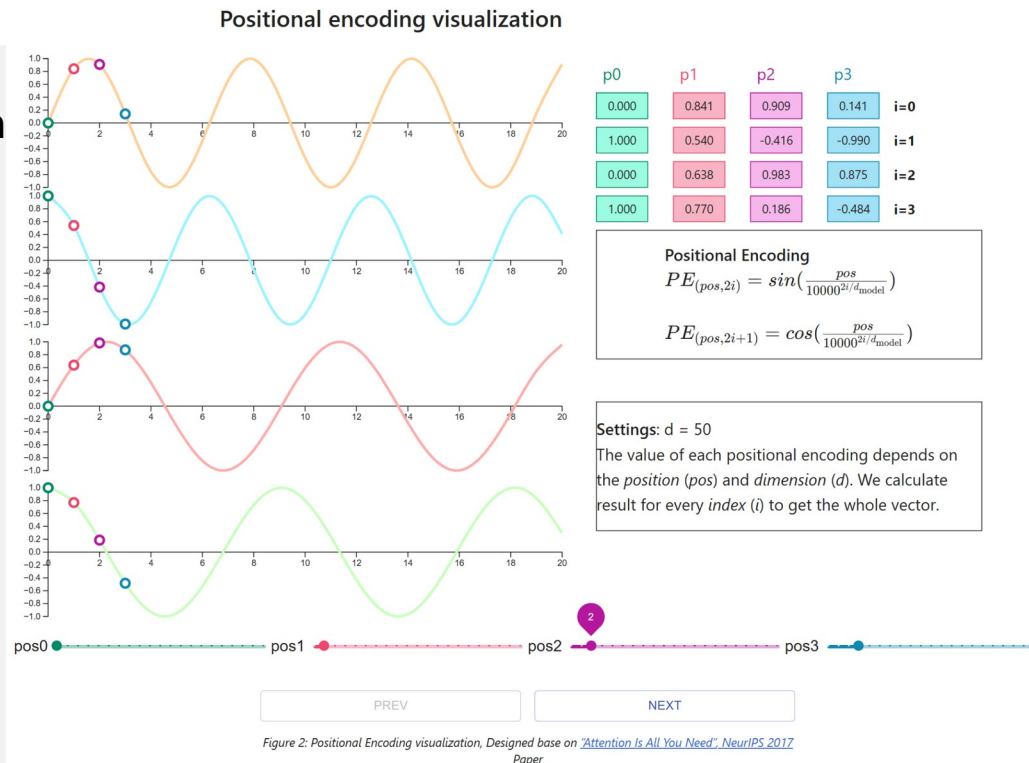


Figure 2: Positional Encoding visualization, Designed base on "Attention Is All You Need". NeurIPS 2017 Paper

Example walkthrough

Checkout this visualization! <https://poloclub.github.io/transformer-explainer/>
(another one that visualize nanoGPT: <https://bbycroft.net/llm>)

Please study transformers after class.

- Another visualization youtube:
<https://www.youtube.com/watch?v=wjZofjX0v4M>

why transformer becomes a mainstream model architecture?

- Easier to capture long-range dependencies
- Easier to parallelize

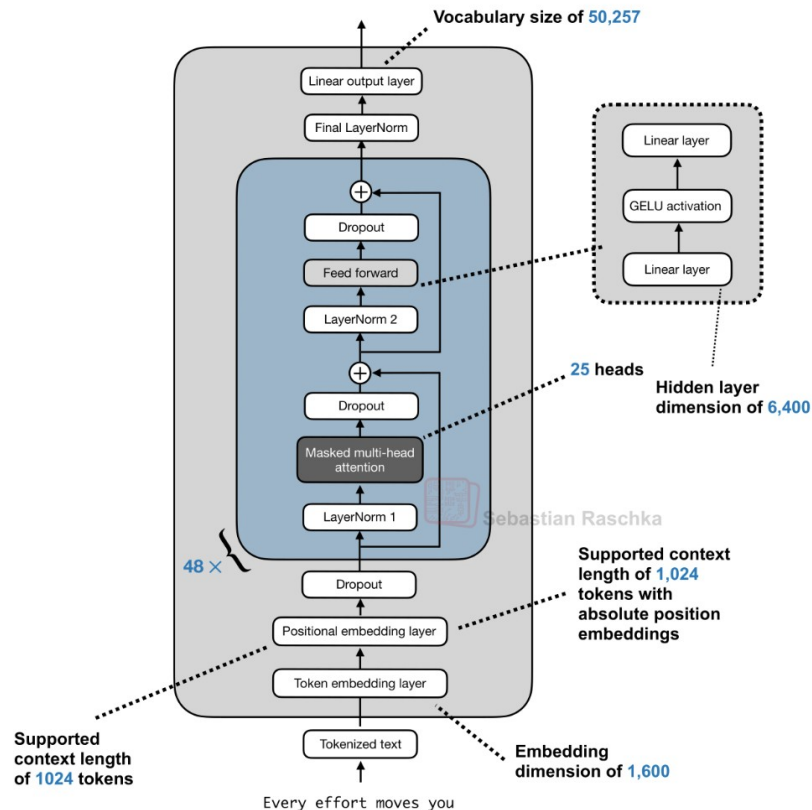
LLM Gallery

Compare LLM architecture.

<https://sebastianraschka.com/llm-architecture-gallery/#architecture-gallery>



GPT-2 XL (1.5B)

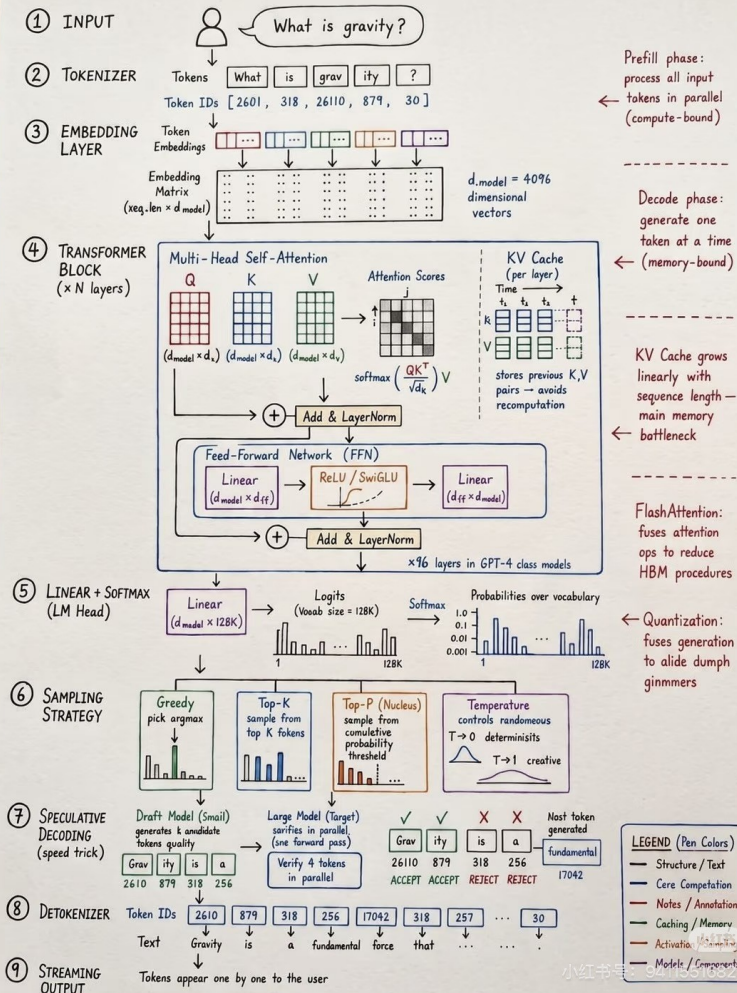


Transformer summary

Source: unknown



How LLMs Actually Generate Text — The Full Inference Pipeline



Greedy decoding vs beam search

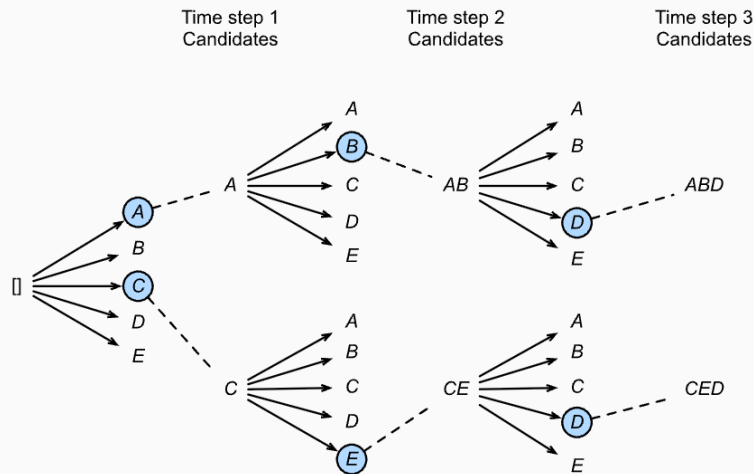


Fig. 10.8.3 The process of beam search (beam size = 2; maximum length of an output sequence = 3). The candidate output sequences are A, C, AB, CE, ABD, and CED.

Source:

https://d2l.ai/chapter_recurrent-modern/beam-search.html

Guided decoding

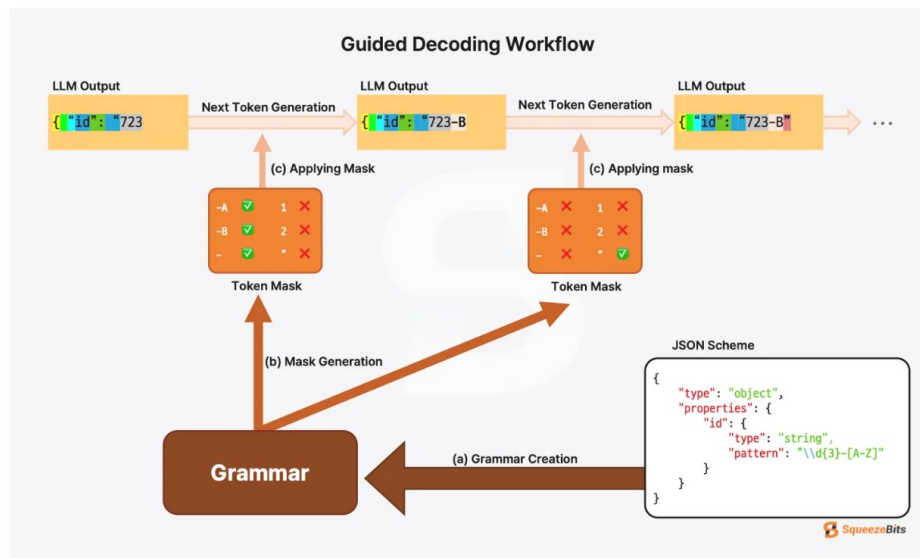


Figure 1. The guided decoding workflow, where a grammar created from a JSON schema generates token masks at each step to constrain the LLM's output.

Source: <https://blog.squeezebits.com/guided-decoding-performance-vllm-sglang>

Guided Decoding in Code Generation

LM decoding+static analysis monitor:

Integrated development environments (IDEs) assist developers in understanding repository context using static analysis. We extend this assistance, enjoyed by developers, to LMs. We propose monitor-guided decoding (MGD) where a monitor uses static analysis to guide

Method to be completed

```
private ServerNode parseServer(String url) {
    Preconditions.checkNotNull(url);
    int start = url.indexOf("http://") + 2;
    int end = url.lastIndexOf("http://") == -1 ?
        url.length() : url.lastIndexOf("http://");
    String str = url.substring(start, end);
    String [] arr = str.split("http://");

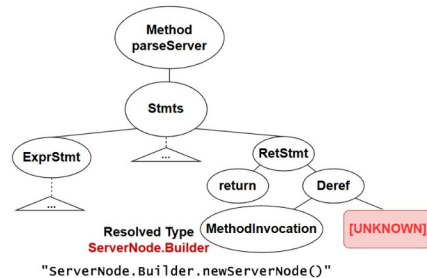
    return ServerNode.Builder
        .newServerNode()
        .build();
}
```

✗ text-davinci-003 and SantaCoder

```
host(arr[0])
.port(Integer.parseInt(arr[1]))
.build();
```

✓ SantaCoder with monitor guided decoding

```
withIp(arr[0])
.withPort(Integer.parseInt(arr[1]))
.build();
```



(a) Example where text-davinci-003 and SantaCoder generate wrong identifiers, but SantaCoder with MGD generates correct identifiers.

(b) Annotated partial AST for the code to the left.

Figure 1: Motivating example to illustrate monitor-guided decoding (MGD).

Guided decoding in Code Generation

Type-Constrained Code Generation with Language Models , PLDI, 2025

<https://arxiv.org/pdf/2504.09246>

LM decoding+type system

	<completion>	Vanilla	Syntax	Types	Description
<code>function is_int(text: string): boolean {</code>	(1) <code>;</code>	accept	reject	reject	Syntactically invalid
<code> const num = Number(text);</code>	(2) <code>ber</code>	accept	accept	reject	Undeclared identifier
<code> return !isNaN(num) &&</code>	(3) <code>()</code>	accept	accept	reject	Disallowed operator
<code> parseInt(num <completion></code>	(4) <code>, 10)</code>	accept	accept	reject	Invalid argument type
	(5) <code>.toString()</code>	accept	accept	accept	Well-formed option

Figure 1. Left is a partial TypeScript program derived from instance #113 of the MBPP benchmark [5], awaiting completion. Right are five completion options: (1)-(4) are invalid and (5) is well-formed. Our type-constrained decoding is the only approach capable of correctly rejecting invalid completions and accepting the valid one.

Algorithm 1 Vanilla LLM-based code generation (without the blue highlights) vs. constrained decoding (with the blue highlights)

Input: LLM, prompt x , completion engine CE_L for language L

Output: Program s such that $s \in L$

```
1: initialize  $s$ 
2: while true do
3:    $v := \text{LLM}(x \circ s)$ 
4:   while true do
5:      $t \sim v$ 
6:     if  $CE_L(s \circ t)$  then break
7:     elif  $t = \text{EOS}$  and  $s \in L$  then break
8:     else  $v[t] := 0$ ; normalize  $v$ 
9:   if  $t = \text{EOS}$  then break
10:   $s := s \circ t$ 
11: return  $s$ 
```

Prompt Engineering

In-context learning

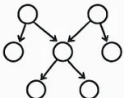
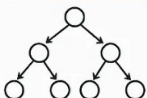
- zero shot
- few shots
- Pros and cons?

Chain of thoughts

Self consistency

Tree of thought

Chain-of-Thought vs Tree-of-Thought vs Self-Consistency:
Prompting Method Performance

METHOD	ILLUSTRATION	DESCRIPTION	PERFORMANCE
Chain-of-Thought		Generate a single reasoning path step-by-step	Foundational method Brittle
Self-Consistency		Sample multiple paths, then select the most frequent answer	Boosts accuracy and robustness
Tree-of-Thought		Explore and evaluate multiple reasoning paths	Excels at complex reasoning

<https://python.plainenglish.io/chain-of-thought-vs-tree-of-thought-vs-self-consistency-prompting-method-performance-77881fa0a02e>

Chain of Thought

Pros and Cons?

Standard Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain of Thought Prompting

Input

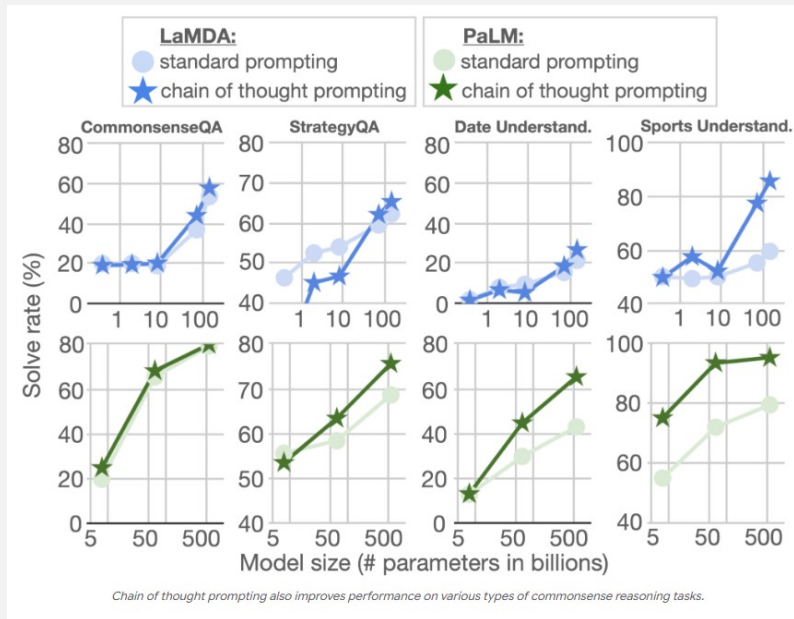
Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

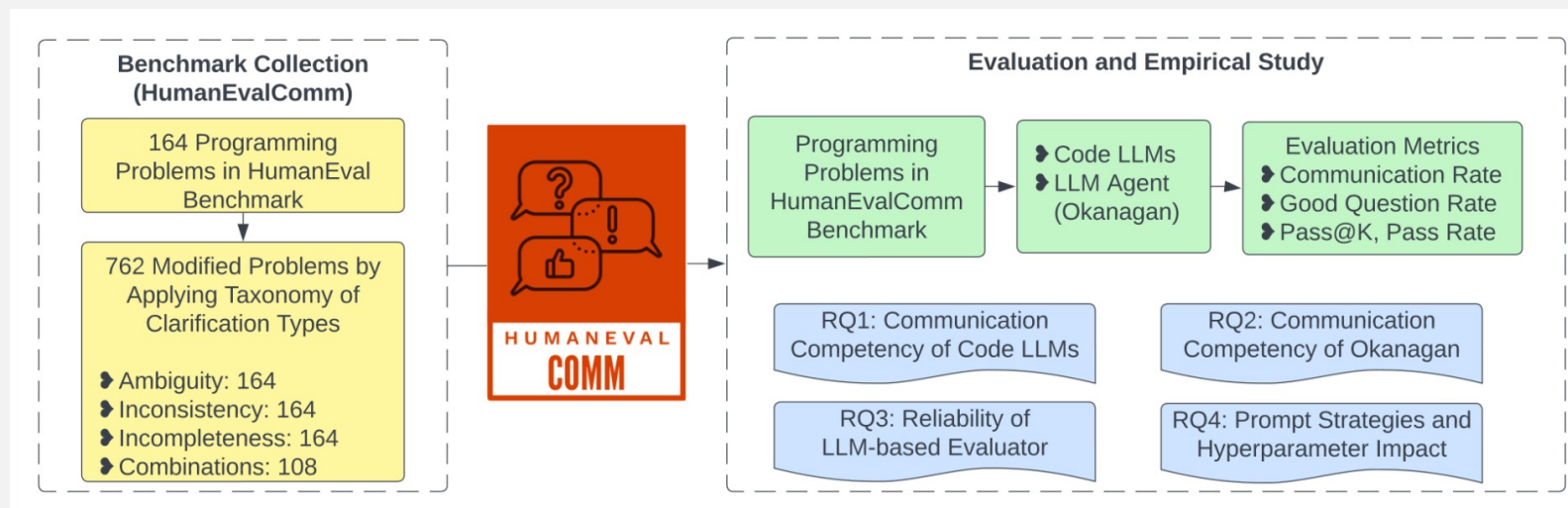


<https://research.google/blog/language-models-perform-reasoning-via-chain-of-thought/>



What if prompt is unclear?

<https://jie-jw-wu.github.io/blog/does-ai-ask.html>



Does self-consistency mean prompt is well-specified?

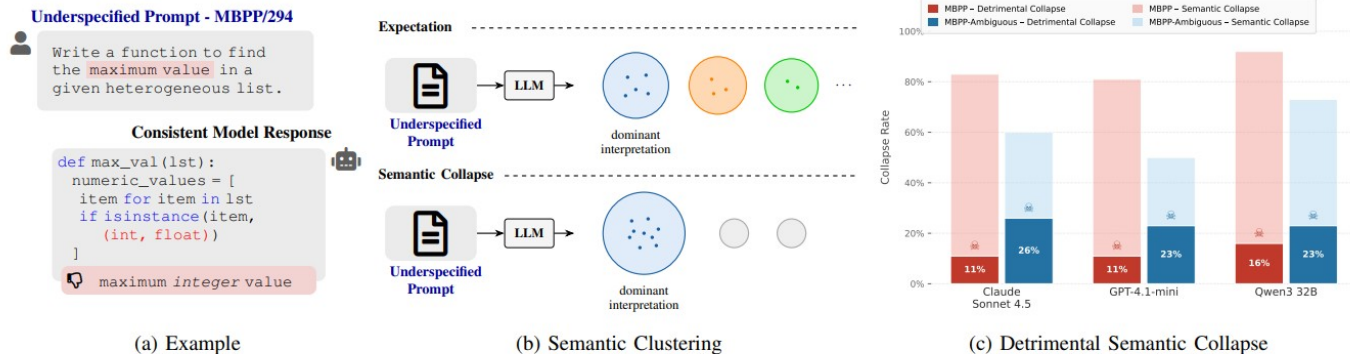


Fig. 1: **Motivating example.** (a) Real-world programming tasks are often underspecified. (b) Semantic clustering approaches assume that underspecified prompts lead LLMs in producing multiple semantically different interpretations when prompted multiple times (with the same prompt/description). We observe that such a phenomenon happens, but more often it does not as LLMs collapse to a single interpretation. (c) Collapse is often detrimental, producing a single incorrect interpretation breaking the key assumption made by semantic clustering methods. For example underspecification manifested in form of ambiguous descriptions, increases the risk for detrimental collapse.

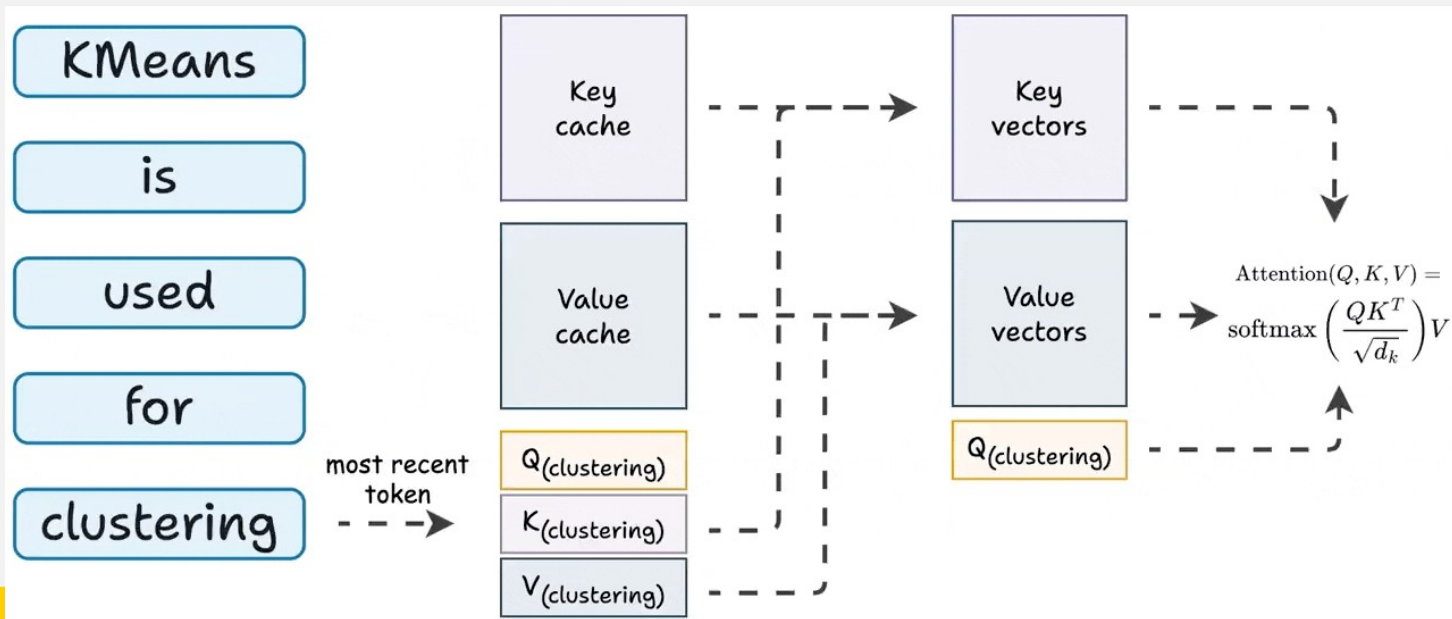
Underspecification does not imply Incoherence: The Risks of Semantic Collapse in Coding Models,

Arxiv <https://arxiv.org/pdf/2607.01953>



KV Cache

<https://www.dailydoseofds.com/p/kv-caching-in-llms-explained-visually/>



BERT

Idea. Keep only encoders + leverage encoded embeddings to make predictions.

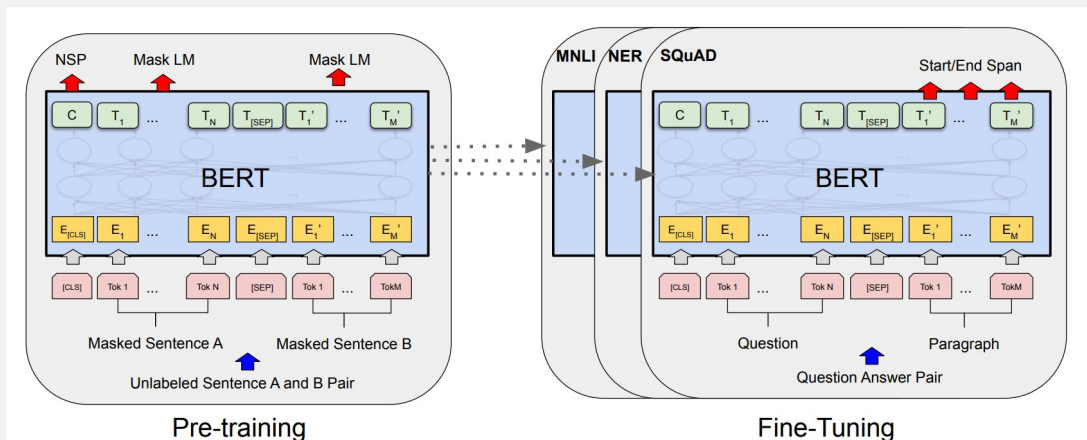


Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. [CLS] is a special symbol added in front of every input example, and [SEP] is a special separator token (e.g. separating questions/answers).

Golden rules in Agentic Coding

Agentic coding guidelines

<https://darrencoxon.substack.com/p/the-golden-rules-of-full-stack-vibe>

<https://dev.to/jackmarchant/agentic-coding-habits-to-accelerate-your-engineering-workflow-4p3g>

Practise Session

Run mini-swe-agent on swe-bench